Contents

SB Connect — Architecture & Technical Documentation

Version: 1.0
Last Updated: July 11, 2026
Stack: React 18 + TypeScript + Vite | NoSQL DB | Cloud Backend

---

1. System Overview

SB Connect is a B2B networking and membership management platform that connects business owners, facilitates referrals, tracks meeting attendance, and provides administrative tools for community management.

Core User Roles

Role	Capabilities
User	Profile creation, request posting/pitching, meeting attendance, chat, deal recording
Admin	All user actions + profile verification, meeting management, notification broadcasting, issue management, deal awarding, data export
Super Admin	All admin actions + admin promotion/demotion, sensitive configuration

Key Value Propositions

- Automated attendance compliance (3 meetings per rolling 6 months)
- End-to-end referral pipeline (Request ⟶ Pitch ⟶ Award ⟶ Deal recording)
- QR-based touchless meeting check-in
- Real-time leaderboard for business given
- Admin audit & compliance reporting

2. Technology Stack

Frontend

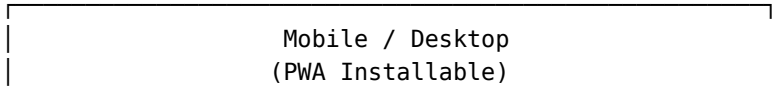
Layer	Technology	Purpose
Framework	**React 18**	UI component library
Language	**TypeScript**	Type safety throughout
Build tool	**Vite**	Fast HMR, optimized production builds
Routing	**React Router v6**	Client-side routing with guards
Styling	**Tailwind CSS**	Utility-first responsive design
Server state	**TanStack React Query v5**	Caching, refetching, loading states
Animation	**Framer Motion**	Page transitions, staggered lists, tilt cards
QR codes	**qrcode.react**	Dynamic QR generation for profiles & meetings
Confetti	**canvas-confetti**	Deal award celebration
PWA	**vite-plugin-pwa**	Service worker, manifest, installability

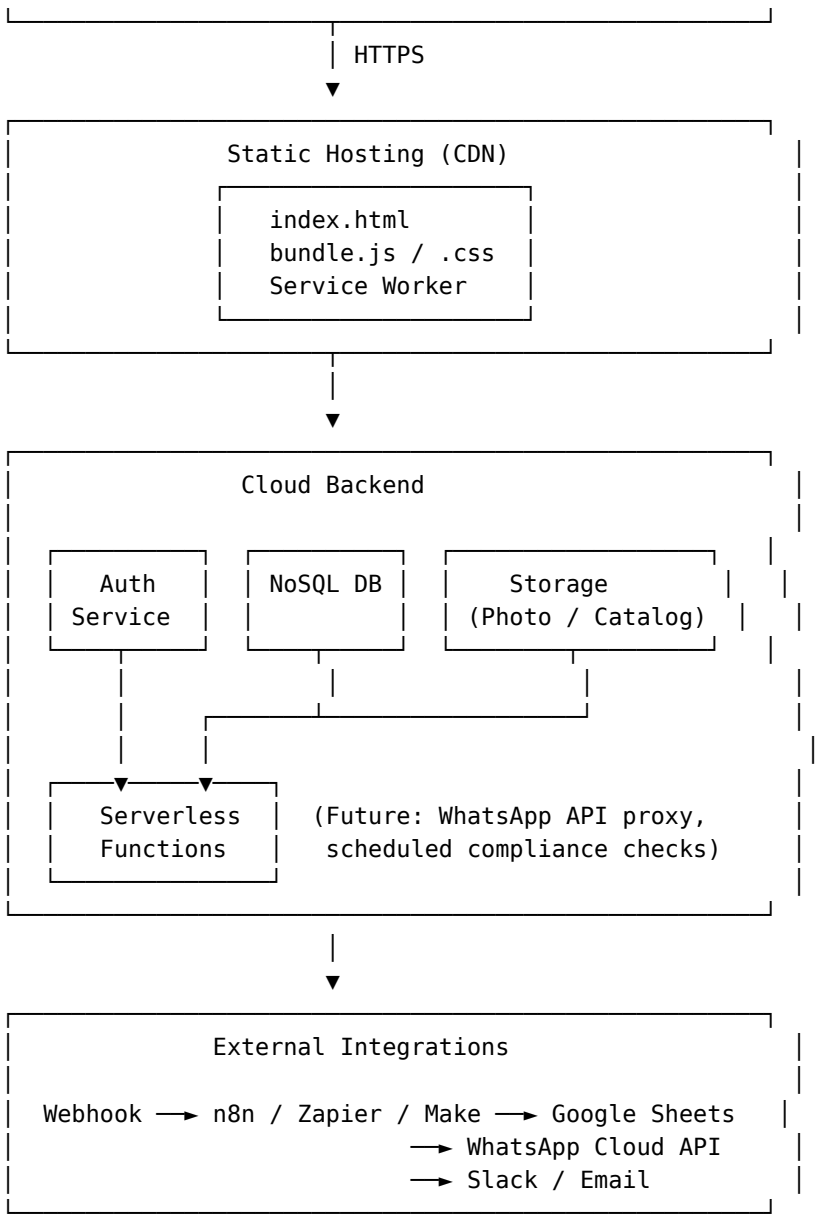
Cloud Backend

Service	Purpose
Auth Service	Email/password authentication, password reset, session management
NoSQL Database	All application data — users, profiles, requests, meetings, chat, etc.
Storage	Profile photos, catalog files (images, PDFs)
Static Hosting	SPA deployment, CDN delivery
Serverless Functions	*(Planned)* WhatsApp API proxy, scheduled tasks

---

3. Architecture Diagram





Data Flow Pattern

All data flows through a unidirectional pattern:

```
User Action → React Component → Service Function → NoSQL DB / Storage
                                                          |
                    UI Update ← React Query Cache ←-------┘
```

Real-time listeners (Auth state, conversations, notifications) use persistent WebSocket connections from the SDK:

```
NoSQL DB ⟶ onSnapshot listener ⟶ React State ⟶ UI update
```

4. Routing & Navigation

Route Table

Path	Page Component	Guard	Purpose
`/login`	Login	Guest	Email/password sign-in
`/register`	Register	Guest	New account creation
`/reset-password`	ResetPassword	Guest	Password reset email
`/seed-admin`	SeedAdmin	Super admin email	First-time admin setup
`/admin-access`	AdminAccess	Auth	Self-service admin upgrade
`/`	Redirects to `/dashboard`	Auth	—
`/dashboard`	Dashboard	Auth	Member home
`/profiles`	Profiles	Auth	Business directory
`/profile/:id`	ProfileDetail	Auth	Individual profile
`/profile/:id/edit`	EditProfile	Auth + Owner/Admin	Profile editing
`/create-profile`	CreateProfile	Auth	New business profile
`/requests`	Requests	Auth	Requests listing
`/requests/create`	CreateRequest	Auth	New request form
`/requests/:id`	RequestDetail	Auth	Request detail + pitch
`/attendance`	Attendance	Auth	Meeting check-in
`/attendance/scan`	AttendanceScan	Auth	QR scan landing
`/chat`	Chat	Auth	Conversation list
`/chat/:id`	ChatDetail	Auth	Individual chat
`/payments`	Payments	Auth	(Placeholder)
`/admin`	Admin	Auth + Admin	Admin panel
`/admin-access`	AdminAccess	Auth	Self-service upgrade

Guard Hierarchy

```
Public Routes (/login, /register, /reset-password)
    │
    ▼
```

```
Auth Guard (checks authenticated user)
    │
    ├── User Routes (all member pages)
    │
    └── Admin Guard (checks role === 'admin' | 'super_admin')
            │
            └── Super Admin Guard (checks role === 'super_admin')
                    └── Admin management, sensitive config
```

Guards are implemented as wrapper components in `src/components/AdminGuard.tsx` and inline checks using `useAuth()`.

---

5. Authentication Flow

Registration Flow

```
User fills form ──► Auth Service createUser ──► Success
                                                    │
                                                    ▼
                                Create 'users' document in NoSQL DB
                                ── uid (from Auth)
                                ── email
                                ── phone
                                ── displayName
                                ── role: 'user'
                                ── onlineStatus: 'online'
                                                    │
                                                    ▼
                                Redirect to /create-profile
```

Login Flow

```
User fills form ──► Auth Service signIn ──► onAuthStateChanged fires
                                                    │
                                                    ▼
                            Subscribe to 'users/{uid}' onSnapshot (real-time)
                            ── Updates AppUser + UserProfile in AuthContext
                                                    │
                                                    ▼
                            Log entry written to 'loginLogs' collection
                                                    │
                                                    ▼
                            Redirect to /dashboard (or ?redirect= param)
```

Role Resolution

Roles are stored in the `users` document. The AuthContext runs a promotion check on every auth state change:

- If user's email matches the hardcoded super admin list (`kktej3d@gmail.com`), role is auto-promoted to `super_admin`

- If email matches the admin list, role is auto-promoted to `admin`
- Normal users remain as `user`

Admin Self-Service Upgrade

The `/admin-access` page provides a 3-step flow:

1. **Request code** — Sends a POST to `adminCodes` collection with a 6-digit code (5-minute expiry)
2. **Verify code** — User enters the code, backend matches it
3. **Activate** — Role updated from `user` to `admin` in the `users` document

---

6. NoSQL Data Model

Collection: `users`

Stores auth-related profile data for every registered user.

Field	Type	Description
`uid`	string	Auth UID (primary key)
`email`	string	Email address
`phone`	string	Phone number
`displayName`	string	Full name
`photoURL`	string	Auth profile photo
`role`	`'user' | 'admin' | 'super_admin'`	Access level
`onlineStatus`	`'online' | 'offline'`	Presence indicator
`lastSeen`	number	Last activity timestamp
`createdAt`	number	Account creation timestamp

Collection: `profiles`

Business profile data for each member. Keyed by the same `uid` as the `users` collection.

Field	Type	Description
`uid`	string	Owner's UID
`ownerName`	string	First name
`ownerSurname`	string	Last name
`phone`	string	Business phone
`companyName`	string	Business name
`categories`	`string[]`	Industry tags (up to 19 options)
`companySize`	string	Employee range
`location`	string	City/area
`contactEmail`	string	Public contact email
`website`	string	Business website URL
`description`	string	Business description (max 500 chars)
`keywords`	`string[]`	Search tags
`photoURL`	string	Profile photo Storage URL
`catalogURLs`	`string[]`	Catalog file URLs (max 5)
`qrCodeURL`	string	Dynamic QR linking to profile
`verified`	boolean	Admin verification status
`membershipStatus`	`'active' | 'inactive' | 'expired'`	Current membership
`membershipExpiry`	number	Expiry timestamp
`membershipDate`	number	When membership started
`editCount`	number	Times profile was edited (max 3)
`locked`	boolean	Profile edit lock (after 3 edits)
`lastRequestsViewedAt`	number	Tracks new-request dots

Collection: `requests`

Business needs and opportunities posted by members.

Field	Type	Description
`id`	string	Auto-generated
`uid`	string	Creator's UID
`companyName`	string	Creator's business name
`title`	string	Request title
`description`	string	Detailed description (max 1000 chars)
`category`	string	From 8 predefined categories
`customCategory`	string	Free-text if category is "Other"
`budget`	string	Budget description
`deadline`	number	Target deadline timestamp
`status`	`'open' | 'closed'`	Current state
`awardedTo`	string	UID of awarded pitcher
`interestCount`	number	Denormalized count of pitches
`interestedUids`	`string[]`	Array of UIDs who pitched
`requesterPhone`	string	Creator's phone for call button
`createdAt`	number	Creation timestamp

Subcollection: `requests/{id}/interests`
Pitches submitted by other members for a specific request.

Field	Type	Description
`requestId`	string	Parent request ID
`uid`	string	Pitcher's UID
`companyName`	string	Pitcher's business name
`phone`	string	Pitcher's phone
`message`	string	Pitch message
`createdAt`	number	Pitch timestamp

Collection: `deals`
Records of business given/awarded (referral tracking).

Field	Type	Description
`id`	string	Auto-generated
`requestId`	string	Related request (if any)
`requestTitle`	string	Denormalized request title
`giverUid`	string	Who gave the business
`giverCompanyName`	string	Giver's business name
`receiverUid`	string	Who received the business
`receiverCompanyName`	string	Receiver's business name
`amount`	string	Deal value (string to support formatting)
`description`	string	Optional deal description
`createdAt`	number	Deal timestamp

Collection: `meetings`

Scheduled networking events.

Field	Type	Description
`id`	string	Auto-generated
`date`	number	Meeting date timestamp
`label`	string	Meeting name/label
`location`	string	Venue/location
`qrCodeURL`	string	QR code for attendance scanning
`active`	boolean	Whether meeting is currently active
`rsvpEnabled`	boolean	Whether RSVP is open
`createdAt`	number	Creation timestamp

Subcollection: `meetings/{id}/rsvps`

RSVP responses from members for a specific meeting.

Field	Type	Description
`meetingId`	string	Parent meeting ID
`uid`	string	Member's UID
`displayName`	string	Member's name
`companyName`	string	Member's business
`response`	`'yes' | 'no' | 'maybe'`	RSVP choice
`respondedAt`	number	Response timestamp

Collection: `attendance`

Meeting attendance records (marked via QR scan or admin).

Field	Type	Description
`id`	string	Auto-generated
`meetingId`	string	Meeting ID attended
`uid`	string	Attendee's UID
`displayName`	string	Attendee's name
`companyName`	string	Attendee's business
`scannedAt`	number	Scan/check-in timestamp

Collection: `conversations`

1-on-1 chat threads between members.

Field	Type	Description
`id`	string	Auto-generated
`participants`	`string[]`	Two participant UIDs
`participantNames`	`Record<string, string>`	UID → display name map
`participantPhotos`	`Record<string, string>`	UID → photo URL map
`lastMessage`	string	Most recent message text
`lastMessageAt`	number	Most recent message timestamp
`lastSenderId`	string	Who sent the last message
`unreadCount`	`Record<string, number>`	UID → unread count map
`createdAt`	number	Conversation creation timestamp

Subcollection: `conversations/{id}/messages`
Individual messages within a conversation.

Field	Type	Description
`senderId`	string	Sender's UID
`text`	string	Message content
`timestamp`	number	Send timestamp
`read`	boolean	Whether recipient has read it

Collection: `notifications`
Broadcast announcements shown to all users in the marquee bar.

Field	Type	Description
`id`	string	Auto-generated
`text`	string	Notification message
`active`	boolean	Whether currently displayed
`createdAt`	number	Creation timestamp

Collection: `userNotifications`
Per-user notifications (issue resolutions, admin messages, pitch alerts).

Field	Type	Description
`id`	string	Auto-generated
`uid`	string	Target user's UID
`type`	`'issue_resolved' | 'admin_message'`	Notification category
`title`	string	Short title
`message`	string	Full notification text
`relatedId`	string	Related entity ID (issue report, request)
`read`	boolean	Whether user has seen it
`createdAt`	number	Creation timestamp

Collection: `issueReports`
User-submitted bug reports and feature requests.

Field	Type	Description
`id`	string	Auto-generated
`uid`	string	Reporter's UID
`userEmail`	string	Reporter's email
`userDisplayName`	string	Reporter's name
`companyName`	string	Reporter's business
`page`	string	Page URL where issue occurred
`subject`	string	Issue summary (max 100 chars)
`description`	string	Detailed description (max 1000 chars)
`status`	`'open' | 'resolved'`	Current state
`adminNote`	string	Admin resolution note
`createdAt`	number	Submission timestamp

Collection: `loginLogs`
Audit trail of all sign-in events.

Field	Type	Description
`uid`	string	User's UID
`email`	string	User's email
`displayName`	string	User's name
`timestamp`	number	Login time

Collection: `config`
App-level configuration (singleton documents).

Document ID	Fields	Description
`webhook`	`{ url, updatedAt }`	External webhook URL for data export

Collection: `adminCodes`
Temporary verification codes for admin self-service upgrade.

Field	Type	Description
`email`	string	Requester's email
`code`	string	6-digit verification code
`expiresAt`	number	5-minute expiry timestamp

Collection: `_health`

Temporary documents for health-check diagnostics.

Field	Type	Description
`timestamp`	number	Health check timestamp

7. State Management

The application uses three state layers:

Layer	Technology	Scope
Auth Context	React Context (`AuthContext.tsx`)	Current user, profile, role, loading state
Server State	TanStack React Query	All NoSQL data with caching, refetching, stale times
Local State	`useState` / `useEffect`	UI state, form inputs, modals, temporary flags

AuthContext

Provides to all descendant components:

```
interface AuthContextValue {
  user: AppUser | null;       // Current auth user
  profile: UserProfile | null; // 'users' collection document
  loading: boolean;            // True during auth initialization
}
```

The `onAuthStateChanged` listener initializes the context. A separate `onSnapshot` listener on `users/{uid}` keeps the profile data in sync in real-time (catches role changes, online status updates, etc.).

React Query Hooks (`src/hooks/useFirebaseQuery.ts`)

Hook	Query Key	Stale Time	Refetch Interval
`useProfiles()`	`['profiles']`	30s	—
`useMeetings()`	`['meetings']`	30s	—
`useLeaderboardQuery()`	`['leaderboard']`	30s	—
`useRequestsQuery()`	`['requests']`	30s	—
`useBusinessProfile(uid)`	`['profile', uid]`	60s	—
`useAttendanceCompliance(uid)`	`['attendance-compliance', uid]`	60s	—
`useUserRSVPs(uid)`	`['rsvps', uid]`	60s	—
`useMeetingRSVPs(meetingId)`	`['meeting-rsvps', meetingId]`	30s	—
`useAllRsvpsByMeeting()`	`['all-rsvps']`	0	15s (admin only)
`useUnverifiedProfiles()`	`['unverified-profiles']`	30s	—
`useTotalBusinessValue()`	`['total-business-value']`	30s	30s

---

8. Component Architecture

Directory Structure

```
src/
├── components/
│   ├── ui/           # Generic UI primitives
│   │   ├── Card.tsx
│   │   ├── Button.tsx
│   │   ├── Badge.tsx
│   │   └── Input.tsx
│   ├── motion/       # Animation wrappers
│   │   ├── AnimatedPage.tsx
│   │   ├── TiltCard.tsx
│   │   ├── StaggerList.tsx
│   │   └── StaggerItem.tsx
│   ├── layout/       # Shell components (or inline in App.tsx)
│   ├── AppLayout.tsx # Main authenticated layout shell
│   ├── Sidebar.tsx / MobileSidebar.tsx
│   ├── TopBar.tsx
│   ├── BottomNav.tsx
│   ├── MarqueeBar.tsx        # Scrolling broadcast notifications
│   ├── StrikeWarning.tsx     # Attendance compliance warning
│   ├── DashboardUpdates.tsx  # Upcoming meetings widget
```

```
│   ├── MembershipCountdown.tsx + FlipCounter.tsx
│   ├── ReportIssue.tsx       # FAB + modal for issue reporting
│   ├── AdminGuard.tsx
│   ├── ErrorBoundary.tsx
│   ├── ProfileSuggestions.tsx
│   └── DashboardUpdates.tsx
├── contexts/
│   └── AuthContext.tsx
├── hooks/
│   ├── useFirebaseQuery.ts   # All React Query hooks
│   └── usePendingVerifications.ts
├── lib/
│   ├── client.ts             # Backend SDK initialization
│   ├── auth.ts               # Auth primitives
│   ├── db.ts                 # NoSQL + Storage operations
│   ├── storage.ts            # File upload utilities
│   ├── format.ts             # Date/currency formatting
│   ├── admin.ts              # Role checking utilities
│   ├── rateLimit.ts          # Client-side rate limiter
│   ├── errorTracker.ts       # LocalStorage error logging
│   ├── auditReport.ts        # Compliance report generation
│   └── healthCheck.ts        # System diagnostics
├── pages/                    # Route-level page components
│   ├── Dashboard.tsx
│   ├── Admin.tsx
│   ├── Requests.tsx
│   ├── RequestDetail.tsx
│   ├── ... (all other pages)
├── types.ts                  # All TypeScript interfaces
└── App.tsx                   # Root component with router
```

Component Hierarchy

```
<App>
  └── <ErrorBoundary>
      └── <Router>
          ├── Guest Routes: Login, Register, ResetPassword
          └── Auth Routes:
              └── <AppLayout>
                  ├── <TopBar>          (date, business value, sign out)
                  ├── <Sidebar>         (desktop nav)
                  ├── <MobileSidebar>   (mobile overlay)
                  ├── <MarqueeBar>      (broadcast notifications)
                  ├── <BottomNav>       (mobile bottom bar)
                  ├── <Outlet>          (page content)
                  ├── <ReportIssue>     (FAB, always mounted)
                  └── <Footer>
```

---

9. Page-by-Page Data Flow

9.1 Dashboard (`/dashboard`)

Section	Data Source	Notes
Stats cards (4)	`myProfile` + `allBusinesses` + `allReqs` + `myNotifications`	Computed inline from React Query data
Strike Warning	`useAttendanceCompliance(uid)`	Conditionally shown
Create Profile CTA	`myProfile == null`	Redirects to `/create-profile`
Quick Actions	Static links	Create Request, View Profile, Record Business
Business Profile	`myProfile` (from AuthContext)	Company info, membership countdown
Leaderboard	`useLeaderboardQuery()`	Top 7 by revenue
Upcoming Meetings	`useMeetings()`	Filtered to future dates, RSVP buttons
Notifications	`getMyNotifications()`	Unread issue resolves + pitches
Deal Form	Local state	Inline modal, writes to `deals` collection

9.2 Business Directory (`/profiles`)

Feature	Implementation
Data	`useProfiles(200)` — all business profiles
Search	Client-side filter by `companyName`, `categories`, `keywords`, `location`
Loading	Skeleton grid with 6 placeholder cards
Navigation	Click card → `/profile/:id`

9.3 Business Profile (`/profile/:id`)

Section	Data Source
Profile info	`getBusinessProfile(uid)`
Online status	`user.onlineStatus` from `users` collection
QR code	`qrcode.react` rendering profile URL
Catalog	`catalogURLs` array → images/PDFs in grid
Edit flow	Inline form → `updateBusinessProfile()` → Storage uploads
Message button	`getOrCreateConversation()` → navigate to `/chat/:id`
Membership editing	Admin-only, updates `membershipStatus` + `membershipExpiry`

9.4 Requests (`/requests`)

Feature	Implementation
My Requests	`getUserRequests(uid)`, filtered from `allReqs`
Open Requests	Filtered to `status === 'open' && uid !== user.uid`
Category filter	Client-side filter on `category` field
Pitch action	`expressInterest()` — transaction + notification to owner
Interest badge	Shows count if `interestCount > 0`
Call button	Shows requester's `requesterPhone` if user already pitched

9.5 Attendance (`/attendance`)

Feature	Implementation
Compliance check	`getAttendanceCompliance(uid)` — counts meetings in last 6 months
Today's meeting	`getActiveMeeting()` — checks if any meeting is happening now
Mark attendance	`markAttendance()` — writes to `attendance` collection
History	`getUserAttendance(uid)` — sorted by date descending

9.6 Chat (`/chat` and `/chat/:id`)

Feature	Implementation
Conversation list	`subscribeToConversations()` — real-time listener
Message thread	`subscribeToMessages(conversationId)` — real-time listener
Send message	`sendMessage()` — writes to `messages` subcollection
Read receipts	`markConversationRead()` — updates `unreadCount`
Auto-cleanup	`deleteOldMessages()` — purges messages older than 30 days

9.7 Payments (`/payments`)

Placeholder page only. Displays: - "Razorpay payment integration coming soon" message - No backend logic implemented yet

---

10. Admin Module

Access Control

The `/admin` route is protected by `<AdminGuard>`, which checks: 1. User is authenticated (`user != null`) 2. User's role is `'admin'` or `'super_admin'`

If unauthorized, redirects to `/admin-access` for self-service upgrade.

Tab System

The admin panel uses a tab-based layout with 6 tabs. Each tab conditionally renders its content:

```
const [activeTab, setActiveTab] = useState<'members' | 'meetings' | 'updates' |
'requests' | 'reports' | 'security'>('members');
```

Tab Details

Tab 1: Members

Panel	Data Source	Actions
Verification Requests	`getUnverifiedProfiles()`	Approve → `verifyBusinessProfile(uid)`
Business Directory	`getAllProfiles()`	CSV export, view profile link
Membership Expiry	`getAllProfiles()`	Color-coded by days remaining, CSV export

Tab 2: Meetings

Panel	Data Source	Actions
Meeting Management	`getMeetings()`	Create → `createMeeting()`, delete → `deleteMeeting()`
Meeting Detail	Selected meeting + `getMeetingAttendance()` + `getMeetingRSVPs()`	Stats grid, QR code, CSV export
Attendance Overview	`useAllRsvpsByMeeting()`	Flat RSVP table across all meetings, CSV export

Tab 3: Updates

Panel	Data Source	Actions
Send Notification	—	Create → `addNotification()`
Notifications List	All notifications	Delete → `deleteNotification()`
Issue Reports	`getIssueReports()`	Resolve (with note) → notifies user; Delete

Tab 4: Requests

Panel	Data Source	Actions
All Requests	`getAllRequests()`	Award Deal (modal) → `awardDeal()`; Close; Delete

Tab 5: Reports

Panel	Data Source	Actions
Audit & Compliance	`auditReport.ts`	Generates JSON with members, attendance, deals, logins → download
System Health	`healthCheck.ts`	NoSQL read/write test, integrity checks, collection counts, error log
Webhook Sync	`config/webhook`	Save URL → `saveWebhookUrl()`; Sync → `triggerWebhookExport()` (POSTs all data)

Tab 6: Security

Panel	Data Source	Actions
Login Activity	`getLoginLogs()`	Table with name/email/time, Refresh
Admin Management	`getAllUsers()` filtered to admins	Add → `setUserRole(email, 'admin')`; Remove → `setUserRole(email, 'user')`

Admin-Only Functions (from `src/lib/db.ts`)

Function	Description
`requireAdmin()`	Throws if caller is not admin
`setUserRole()`	Promote/demote users
`verifyBusinessProfile()`	Mark profile as verified
`awardDeal()`	Award contract + close request
`createMeeting()`	New meeting event
`deleteMeeting()`	Remove meeting
`deleteNotification()`	Remove broadcast notification
`resolveIssueReport()`	Mark issue as resolved (with note)
`deleteIssueReport()`	Remove issue report
`saveWebhookUrl()` / `getWebhookUrl()`	Webhook config
`triggerWebhookExport()`	POST all collections to webhook URL
`getLoginLogs()`	View login audit trail
`deleteOldMessages()`	Purge messages > 30 days

---

11. Real-time Subscriptions

Subscription	Method	Location	Purpose
Auth state	`onAuthStateChanged`	`AuthContext.tsx`	Detect login/logout
User profile	`onSnapshot` on `users/{uid}`	`AuthContext.tsx`	Real-time role/status updates
Conversations	`onSnapshot` on `conversations`	`Chat.tsx`	Live conversation list
Messages	`onSnapshot` on `conversations/{id}/messages`	`ChatDetail.tsx`	Live message thread
Broadcast notifications	`onSnapshot` on `notifications`	`MarqueeBar.tsx`	Live scrolling updates
Meetings	`onSnapshot` on `meetings`	`DashboardUpdates.tsx`	Upcoming meetings widget

All subscribers are cleaned up via `onUnsubscribe()` returned by the SDK's `onSnapshot` to prevent memory leaks.

---

12. Notification System

Two Notification Channels

Channel 1: Broadcast (`notifications` collection)

- Created by admins via the "Send Update" panel in Admin tab 3
- All authenticated users see them in the `MarqueeBar` (scrolling text at top of layout)
- Real-time via `onSnapshot` listener
- Active flag controls visibility

Channel 2: User-Specific (`userNotifications` collection)

- Per-user notifications created programmatically
- Types:
  - `issue_resolved` — When admin resolves a user's issue report
  - `admin_message` — When someone pitches on a user's request
- Displayed on the Dashboard in the Notifications block
- Read/unread state tracked per notification

Issue Resolution Flow

```
User submits issue ──► Admin sees in Admin tab 3 panel
                                  │
                                  ▼
                       Admin types note + clicks Resolve
                                  │
                                  ├── resolveIssueReport(id, note)
                                  │   └── Updates issueReports/{id} (status →
'resolved', adminNote)
                                  │
                                  └── sendUserNotification(uid, 'issue_resolved', ...)
                                      └── Creates userNotifications document
                                                  │
                                                  ▼
                                      User's Dashboard shows new notification
                                      with green "Resolved" pill
```

Pitch Notification Flow

```
User pitches on request ──► expressInterest() transaction
                                        │
                                        ├── Updates request (interestCount++,
interestedUids)
                                        ├── Creates interests subcollection document
                                        └── sendUserNotification(ownerUid,
'admin_message', ...)
                                            └── Creates userNotifications document
                                                        │
                                                        ▼
                                            Request owner's Dashboard shows
```

new notification with blue "New Pitch" pill

---

13. Webhook Integration

Configuration

The webhook URL is stored in the `config` collection as a singleton document `config/webhook`:

```
interface WebhookConfig {
  url: string;       // Target URL (e.g., Google Apps Script, n8n, Zapier)
  updatedAt: number; // Last update timestamp
}
```

Data Export

The `triggerWebhookExport()` function reads all 9 collections and POSTs a JSON payload:

```
interface WebhookPayload {
  exportedAt: number;
  exportedBy: string;    // Admin UID
  users: UserProfile[];
  profiles: BusinessProfile[];
  meetings: Meeting[];
  requests: Request[];
  deals: Deal[];
  attendance: Attendance[];
  notifications: AppNotification[];
  loginLogs: LoginLog[];
  issueReports: IssueReport[];
}
```

Use Cases

- **Google Sheets** — Send to Google Apps Script webhook URL to populate a spreadsheet
- **n8n / Make / Zapier** — Trigger workflows for CRM sync, email notifications, WhatsApp broadcasts
- **Data Backup** — Export all data to an external storage system
- **Analytics** — Pipe data into a BI tool or data warehouse

---

14. PWA Configuration

Configured via `vite-plugin-pwa` in `vite.config.ts`:

```
VitePWA({
  registerType: 'autoUpdate',
  manifest: {
    name: 'SB Connect',
    short_name: 'SB Connect',
    description: 'B2B Networking & Membership Platform',
    theme_color: '#2A11A6',
```

```
    background_color: '#FAF8F5',
    display: 'standalone',
    orientation: 'portrait',
    icons: [
      { src: 'icons/icon-192x192.png', sizes: '192x192', type: 'image/png' },
      { src: 'icons/icon-512x512.png', sizes: '512x512', type: 'image/png' },
    ],
  },
  workbox: {
    globPatterns: ['**/*.{js,css,html,ico,png,svg,webmanifest}'],
  },
})
```

PWA Features

Feature	Implementation
Install prompt	Browser native `beforeinstallprompt` event
Auto-update	`registerType: 'autoUpdate'` — updates service worker on page reload
Offline shell	Service worker precaches all static assets
Splash screen	Platform-native splash using manifest colors + icons
App shortcuts	*(Not configured)*

---

15. Database Security Rules

All rules are defined in `firestore.rules` and follow these principles:

Principle	Description
Authenticated access minimum	No unauthenticated reads or writes
Owner-scoped writes	Users can only write their own documents
Admin override	Admins can read/write any collection
Subcollection inheritance	Subcollections inherit parent collection rules with additional access
Least privilege	Each rule allows only the minimum required access

Collection-Level Rules Summary

Collection	Read	Create	Update	Delete
`users`	All auth	—	Owner	Admin
`profiles`	All auth	Owner	Owner, Admin	Admin
`requests`	All auth	All auth	Owner, Admin	Admin
`requests/{id}/interests`	All auth	All auth	—	Admin
`meetings`	All auth	Admin	Admin	Admin
`meetings/{id}/rsvps`	All auth	All auth	Owner	Admin
`attendance`	All auth	All auth	Admin	Admin
`deals`	All auth	All auth	Admin	Admin
`issueReports`	Admin	All auth	Admin	Admin
`notifications`	All auth	Admin	Admin	Admin
`userNotifications`	Owner	Admin	Owner	Admin
`conversations`	Participant	All auth	Participant	Admin
`conversations/{id}/messages`	All auth	All auth	Sender	Admin
`loginLogs`	Admin	All auth	—	Admin
`config`	Admin	—	Admin	—

---

16. End-to-End Data Flows

Flow 1: User Onboarding → Profile Creation

```
1. User visits /register
2. Fills form (name, email, phone, password)
3. Auth Service creates account → onAuthStateChanged fires
4. 'users/{uid}' document created (role: 'user')
5. Redirect to /create-profile
6. Fills business profile form (company, categories, photo, etc.)
7. Photo uploaded to Storage → URL saved in profile
8. 'profiles/{uid}' document created
9. Redirect to /dashboard
```

Flow 2: Meeting Attendance via QR

```
1. Admin creates meeting in Admin tab 2
   └── 'meetings/{id}' document created
   └── QR code generated with URL /attendance/scan?meetingId={id}

2. Member scans QR code on phone
   └── Opens /attendance/scan?meetingId={id}
   └── Auth check → if not logged in, redirect to /login with ?redirect= param
```

```
   └── Auto-marks attendance via markAttendance(meetingId, uid)
   └── Shows success/error screen
   └── Redirects to /attendance

3. Admin views attendance in Admin tab 2 detail panel
   └── getMeetingAttendance(meetingId) → table of attendees
```

Flow 3: Request → Pitch → Deal Award

```
1. Member A creates a request at /requests/create
   └── 'requests/{id}' document created (status: 'open')

2. Member B sees the request in Open Requests list at /requests
   └── Clicks Pitch button
   └── expressInterest() transaction:
       ├── Reads request document (validates not already pitched)
       ├── Updates: interestCount++, interestedUids push
       ├── Creates 'requests/{id}/interests/{interestId}'
       └── sendUserNotification(ownerUid, 'admin_message', ...)

3. Member A sees notification on Dashboard
   └── "New Pitch" blue pill with company name

4. Member A visits /requests/:id
   └── Gets interests list
   └── Clicks "Award Contract" for Member B
   └── Award form → enters amount → confirms
   └── awardDeal() creates 'deals/{id}' and closes request

5. Leaderboard updates
   └── Deal revenue + amount for Member B and Member A
   └── Confetti animation on screen
```

Flow 4: Issue Report → Resolution

```
1. User clicks Report Issue FAB (always visible bottom-right)
   └── Modal opens → fills subject + description → clicks Submit
   └── 'issueReports/{id}' created (status: 'open')

2. Admin sees in Admin tab 3 → Issue Reports panel
   └── List of open reports with subject, reporter, page, date

3. Admin types resolution note → clicks Resolve
   └── resolveIssueReport(id, note) updates document
   └── sendUserNotification(uid, 'issue_resolved', ...)
       └── 'userNotifications/{id}' created

4. User sees on Dashboard next load
   └── Notifications stat card shows unread count
   └── Notifications block shows green "Resolved" pill
   └── Subject and admin note visible
```

17. Performance & Optimization

Client-Side

Technique	Implementation
React Query caching	`staleTime: 30-60s` prevents redundant reads on route changes
Polling intervals	`refetchInterval: 15-30s` for admin panels needing freshness
Skeleton loading	Placeholder UI rendered while data fetches
Lazy routes	All page components loaded lazily via `React.lazy()`
Code splitting	Vite manual chunks in `vite.config.ts` split vendor, SDK, and QR dependencies
Debounced search	Client-side directory search filters on every keystroke
Optimistic updates	Pitch interest count updated locally before server confirms
Error boundaries	`ErrorBoundary.tsx` catches render errors with retry
Error tracking	`errorTracker.ts` logs runtime errors to LocalStorage (200 max)

Bundle Size (Production)

Chunk	Size (gzipped)	Contents
`vendor-*.js`	~72 KB	React, React Router, TanStack Query, Framer Motion
`firebase-*.js`	~177 KB	Backend SDK (auth, DB, storage)
`index-*.js`	~100 KB	Application code
`qr-*.js`	~9 KB	QR code library (lazy loaded)

Data Optimization

Practice	Detail
Denormalized counts	`interestCount` on request avoids subcollection count queries
Array indexes	`interestedUids` enables client-side pitch detection without subcollection reads
Query limits	All collection queries use reasonable limits (default 100-200)
Real-time sparing	Only 6 `onSnapshot` listeners active app-wide
Batch operations	Webhook export reads all collections in parallel via `Promise.all`

End of Architecture Document